

# Learning to Branch: Accelerating Resource Allocation in Wireless Networks

Mengyuan Lee, Guanding Yu , *Senior Member, IEEE*, and Geoffrey Ye Li, *Fellow, IEEE*

**Abstract**—Resource allocation in wireless networks, such as device-to-device (D2D) communications, is usually formulated as mixed integer nonlinear programming (MINLP) problems, which are generally NP-hard and difficult to get the optimal solutions. Traditional methods to solve these MINLP problems are all based on mathematical optimization techniques, such as the branch-and-bound (B&B) algorithm that converges slowly and has forbidding complexity for real-time implementation. Therefore, machine learning (ML) has been used recently to address the MINLP problems in wireless communications. In this paper, we use imitation learning method to accelerate the B&B algorithm. With invariant problem-independent features and appropriate problem-dependent feature selection for D2D communications, a good auxiliary prune policy can be learned in a supervised manner to speed up the most time-consuming branch process of the B&B algorithm. Moreover, we develop a mixed training strategy to further reinforce the generalization ability and a deep neural network (DNN) with a novel loss function to achieve better dynamic control over optimality and computational complexity. Extensive simulation demonstrates that the proposed method can achieve good optimality and reduce computational complexity simultaneously.

**Index Terms**—Machine learning, device-to-device communications, resource allocation, mixed integer nonlinear programming, imitation learning, branch-and-bound algorithm.

## I. INTRODUCTION

TO MEET with the increasing demand for higher data rate and better QoS requirements, device-to-device (D2D) communications have been proposed as a promising technique for the LTE-Advanced networks and have recently attracted significant attention from both academic and industry communities [1]–[5]. The key idea of D2D communications is enabling proximity users to communicate directly by reusing the wireless spectrum of conventional cellular users (CUs). Compared with traditional cellular communications, this new technique can significantly reduce the traffic load in the core network and increase the network throughput as well. However, D2D communications

may generate severe interference to the existing CUs due to spectrum reusing. Proper resource allocation can improve the system performance while reducing the co-channel interference between D2D and CUs.

Resource allocation for D2D communications and other wireless networks is usually formulated as mixed integer nonlinear programming (MINLP) problems, which are in general NP-hard and no efficient global optimal algorithm is available yet. Existing solutions to those MINLP problems are usually based on various mathematical optimization techniques. In general, the optimal solutions to the MINLP problems can only be achieved by the branch-and-bound (B&B) algorithm [6]. However, the worst-case computational complexity of the B&B algorithm is exponential, causing it impractical for real-time implementation. Therefore, some works aim to develop near-optimal or sub-optimal algorithms and reduce computational complexity, such as methods based on game theory [7], [8] or graph theory [9], [10]. Heuristic algorithms, which alternatively or iteratively update the combinatorial and continuous variables, have also been widely investigated [11], [12]. The sub-optimal methods usually suffer from two major shortcomings. On the one hand, the performance gaps between the sub-optimal solutions and the optimal ones are hard to control. On the other hand, many iterative based heuristic algorithms still have high computational complexity for real-time implementation although they are faster than the B&B algorithm.

The aforementioned two disadvantages can be overcome by ML techniques. Its inference stage is generally very fast and we can adopt different techniques to improve and control the output performance. Therefore, this paper incorporates the machine learning (ML) technique and mathematical optimization technique to address the resource allocation in D2D communications, which can be also applied to resource allocation in other wireless networks. We are inspired by recently emerged studies on applying the ML techniques to address mathematical optimization problems in wireless communications [13]–[19]. In [13], [14], deep neural networks (DNNs) have been utilized to deal with the power control problems in wireless networks. In [15]–[17], reinforcement learning has been adopted to solve resource allocation problems for various wireless networks. In [18], spatial learning has been developed to schedule interfering links in D2D networks without the need of channel state information (CSI). In [19], linear sum assignment problems (LSAPs), which are often encountered in wireless communications, have been solved based on DNNs. All the above studies follow the end-to-end learning paradigm [13]

Manuscript received September 4, 2019; revised November 2, 2019; accepted November 12, 2019. Date of publication November 15, 2019; date of current version January 15, 2020. This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFB1802302 and in part by the Fundamental Research Funds for the Central Universities. This article was presented in part at IEEE VTC 2019-Fall, Honolulu, HI, USA, Sep. 2019. The review of this article was coordinated by Prof. Xianbin Wang. (Corresponding author: Guanding Yu.)

M. Lee and G. Yu are with the Zhejiang Provincial Key Laboratory of Information Processing, Communication and Networking, Zhejiang University, Hangzhou 310027, China (e-mail: mengyuan\_lee@zju.edu.cn; yuguanding@zju.edu.cn).

G. Y. Li is with the School of ECE, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: liye@ece.gatech.edu).

Digital Object Identifier 10.1109/TVT.2019.2953724

and treat a given resource optimization problem as a “black box” to learn its input/output relation by various ML techniques. This end-to-end paradigm is very suitable for optimization problems with only one kind of output variables. For examples, problems in [13]–[15] only include continuous output variables, whereas [18], [19] only involve combinatorial ones. However, such end-to-end paradigm is hard to effectively solve the MINLP problems due to the more complicated algorithm structures.

Therefore, this paper exploits the specific algorithm structures to solve the MINLP problems for resource allocation in D2D communications using ML technique. Specifically, we formulate a mathematical problem to maximize the minimum data rate among D2D pairs by jointly optimizing the channel allocation and power control. Instead of directly using mathematical techniques to solve it, we turn to a hybrid ML and mathematical method. We propose to leverage the imitation learning to reduce the computational complexity of the B&B algorithm by accelerating the most time-consuming branch process. The imitation learning has been first proposed in [20] for the mixed integer linear programming (MILP) problems. Although this very approach has been introduced in wireless communications in [21] to solve the resource allocation problems in cloud radio access networks (Cloud-RANs), there is no work applying the imitation learning to solve D2D resource allocation problems.

Imitation learning can be further converted into a binary classification problem with proper feature design, which can be solved by the classical support vector machine (SVM) in a supervised manner. To improve the learning performance, we carefully select the problem-dependent features for the resource allocation problems in D2D communications in addition to the problem-independent features in [20]. The problem-dependent features include the power constraint of each D2D link as well as the CSI of both D2D and interference links. We further investigate the influence of the problem-dependent features in terms of optimality and computational complexity.

The ML based algorithm is expected to have few needed training samples, fast training speed, and good generalization ability. Therefore, we make the following three major improvements on the specific learning algorithm as compared with the work in [20], [21]. First, we simplify the learning task without performance loss by learning an auxiliary prune policy to prune the non-optimal nodes that are not fathomed. This improvement leads to fewer needed training samples and faster training speed than the existing one that discards the original prune policy in the B&B algorithm and learns a new one to prune all the non-optimal nodes. Second, we propose to only collect the nodes that are visited under current policy and are not fathomed by the original B&B prune policy to further accelerate the training process. This modification can also mitigate the dataset imbalance issue and improve the accuracy of the binary classifier. Third, we develop a mixed training strategy to reinforce the generalization ability of the imitation learning method. A novel loss function is also developed when utilizing the DNN as the binary classifier to dynamically control the optimality and computational complexity. By extensive simulation results, we verify the effectiveness of

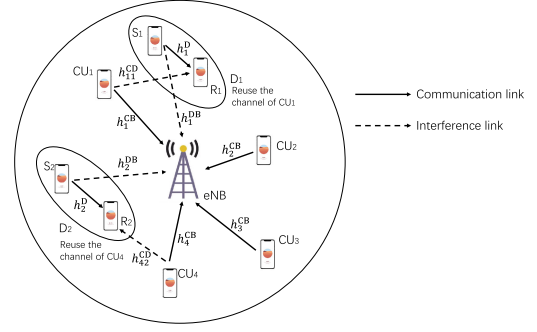


Fig. 1. System model.

the proposed method for D2D communications and highlight some insights about practical implementation as well.

The rest of this paper is organized as follows. In Section II, we formulate resource allocation into an MINLP problem. In Section III, we will transform the problem into a more tractable one and develop the optimal B&B algorithm to solve it. Section IV introduces the imitation learning method to accelerate the B&B algorithm. In Section V, we present test results of the proposed method, which motivate us to make two further improvements in Section VI. Finally, we will conclude this paper in Section VII.

## II. PROBLEM FORMULATION

In this section, we formulate the resource allocation in wireless networks into an MINLP problem using D2D communications as an example.

### A. Resource Allocation in D2D Communications

As depicted in Fig. 1, we consider an uplink single-cell system with  $K$  CUs in a set  $\mathcal{K} = \{1, \dots, K\}$  and  $L$  D2D pairs in a set  $\mathcal{L} = \{1, \dots, L\}$ . We assume that each uplink CU connects to the Evolved NodeB (eNB) with an orthogonal channel. Moreover, we assume that D2D pairs transmit data by reusing the uplink channels of CUs. In the D2D networks, the number of D2D pairs is usually smaller than that of cellular users [22], [23]. Therefore, we assume  $K \geq L$  in this paper. Nevertheless, our proposed method can also be utilized while  $K \leq L$ .

As in Fig. 1, we denote  $h_{kl}^{CD}$  as the instantaneous channel power gain of the interference link between CU  $k$  and the receiver of D2D pair  $l$ ,  $h_k^{CB}$  as the channel power gain between CU  $k$  and the eNB,  $h_l^D$  as the channel power gain between D2D pair  $l$ , and  $h_l^{DB}$  as the channel power gain of the interference link between the transmitter of D2D pair  $l$  and the eNB.

We further introduce  $\rho = [\rho_{kl}]$  as the indicator vector of the channel allocation. Specifically,  $\rho_{kl} = 1$  if the channel of CU  $k$  is reused by D2D pair  $l$ , and  $\rho_{kl} = 0$  otherwise. Denote  $\mathbf{p}^C = [p_k^C]$  and  $\mathbf{p}^D = [p_{kl}^D]$  as the transmit power vectors for  $K$  CUs and  $L$  D2D pairs, respectively, where  $p_k^C$  denotes the allocated transmit power for CU  $k$  and  $p_{kl}^D$  denotes the power of D2D pair  $l$  on the channel of CU  $k$ .

Since each CU channel is assumed to be reused by at most one D2D pair, the signal-to-interference-plus-noise ratio (SINR) of

D2D pair  $l$  on the channel of CU  $k$  can be written as

$$SINR_{kl}^D(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}) = \frac{\rho_{kl} p_{kl}^D h_l^D}{\sigma_N^2 + p_k^C h_{kl}^{CD}},$$

where  $\sigma_N^2$  denotes the power of the additive white Gaussian noise (AWGN). Similarly, the SINR achieved by CU  $k$  can be expressed as

$$SINR_k^C(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}) = \frac{p_k^C h_k^{CB}}{\sigma_N^2 + \sum_{l \in \mathcal{L}} \rho_{kl} p_{kl}^D h_l^{DB}}.$$

Accordingly, the data rates in bits per second per hertz (i.e., normalized by the channel bandwidth) of CU  $k$  and D2D pair  $l$  on all channels can be written as

$$R_k^C(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}) = \log(1 + SINR_k^C(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho})), \quad (1)$$

$$\begin{aligned} R_l^D(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}) &= \sum_{k \in \mathcal{K}} \rho_{kl} R_{kl}^D(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}) \\ &= \sum_{k \in \mathcal{K}} \rho_{kl} \log(1 + SINR_{kl}^D(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho})), \end{aligned} \quad (2)$$

respectively, where  $R_{kl}^D$  is the data rate of D2D pair  $l$  on the channel of CU  $k$ .

Resource allocation in D2D communications is to determine  $\mathbf{p}^C$ ,  $\mathbf{p}^D$ , and  $\boldsymbol{\rho}$  to optimize the overall network performance.

### B. Problem Formulation

To achieve fairness among different users, we specifically consider a resource allocation problem to maximize the minimum data rate of D2D pairs in this paper with the following constraints. First, the minimum data rate of each CU is required to be no less than given thresholds. Second, the power of individual links is also constrained. Finally, as mentioned above, each channel can be reused by at most one D2D pair to limit the interference between different D2D pairs. Therefore, the resource allocation problem can be mathematically formulated as

$$\max_{\{\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}\}} \min_{l \in \mathcal{L}} R_l^D(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}), \quad (3)$$

subject to

$$\rho_{kl} \in \{0, 1\}, \quad \forall k \in \mathcal{K}, l \in \mathcal{L}, \quad (3a)$$

$$\sum_{l \in \mathcal{L}} \rho_{kl} \leq 1, \quad \forall k \in \mathcal{K}, \quad (3b)$$

$$\sum_{k \in \mathcal{K}} \rho_{kl} p_{kl}^D \leq P_{\max}^D, \quad \forall l \in \mathcal{L}, \quad (3c)$$

$$R_k^C(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}) \geq R_{\min}^C, \quad \forall k \in \mathcal{K}, \quad (3d)$$

$$p_k^C \leq P_{\max}^C, \quad \forall k \in \mathcal{K}, \quad (3e)$$

where  $R_{\min}^C$  is the minimum rate of CUs,  $P_{\max}^C$  and  $P_{\max}^D$  are the maximum transmit power level of CUs and D2D pairs, respectively. Note that, the proposed algorithm can be extended to other resource allocation problems with different objective functions since the B&B algorithm is a general approach to solve MINLP problems.

Problem (3) is a typical resource allocation problem in D2D communications. Similar resource allocation problems can be also found in other wireless networks. It is an MINLP problem, which is NP-hard in general. The B&B algorithm can find its optimal solution but with high computational complexity [6]. In the next section, we will focus on developing the B&B algorithm for Problem (3) as the preparation for aforementioned ML-based acceleration process.

### III. OPTIMAL BRANCH-AND-BOUND ALGORITHM

The B&B algorithm is widely used for MINLP problems. However, most existing studies on resource allocation have focused on heuristic algorithms and only used the B&B algorithm as the baseline without detailed development due to its high computational complexity. In this paper, we will accelerate the B&B algorithm with the help of an ML technique, thus developing the optimal B&B algorithm in detail is necessary. Therefore, in this section, we will first transform Problem (3) into a more tractable one and then develop the optimal B&B algorithm to solve it.

#### A. Problem Transformation

To transform Problem (3) into a more tractable one, we first derive a proposition about the optimal power allocation of D2D pair  $l$  on the channel of CU  $k$  in the following, which is proved in Appendix A.

*Proposition 1:* If D2D pair  $l$  is allowed to reuse the channel of CU  $k$ , then the optimal power of CU  $k$  can be written as

$$p_k^C = \frac{(2^{R_{\min}^C} - 1)(\sigma_N^2 + p_{kl}^D h_l^{DB})}{h_k^{CB}},$$

where  $p_{kl}^D \in [0, p_{kl}^{\max}]$  and

$$p_{kl}^{\max} = \min\{(1/h_l^{DB})(p_{\max}^C h_k^{CB}/(2^{R_{\min}^C} - 1) - \sigma_N^2), P_{\max}^D\}.$$

Based on Proposition 1 and the derivation in Appendix B, we can rewrite the data rate of D2D pair  $l$  on all channels as a function of  $\{\mathbf{p}^D, \boldsymbol{\rho}\}$  as stated in the following.

*Proposition 2:* The data rate of D2D pair  $l$  on all channels defined in (2) can be rewritten into a function of  $\mathbf{p}^D$  and  $\boldsymbol{\rho}$  as

$$R_l^D(\mathbf{p}^C, \mathbf{p}^D, \boldsymbol{\rho}) = \sum_{k \in \mathcal{K}} \log \left( 1 + \frac{\rho_{kl} p_{kl}^D}{a_{kl} + b_{kl} p_{kl}^D} \right) \triangleq \hat{R}_l^D(\mathbf{p}^D, \boldsymbol{\rho}),$$

where  $a_{kl} \triangleq \sigma_N^2/h_l^D + (2^{R_{\min}^C} - 1)h_{kl}^{CD}\sigma_N^2/h_l^D h_k^{CB}$  and  $b_{kl} \triangleq (2^{R_{\min}^C} - 1)h_{kl}^{CD}h_l^{DB}/h_l^D h_k^{CB}$ .

From the above discussion, Problem (3) is equivalent to the following optimization problem

$$\max_{\{\mathbf{p}^D, \boldsymbol{\rho}\}} \min_{l \in \mathcal{L}} \hat{R}_l^D(\mathbf{p}^D, \boldsymbol{\rho}), \quad (4)$$

subject to (3a), (3b), (3c), and

$$p_{kl}^D \leq p_{kl}^{\max}, \quad \forall k \in \mathcal{K}, l \in \mathcal{L}. \quad (4a)$$

#### B. Optimal Branch-and-Bound Algorithm

We propose to use the B&B algorithm to solve Problem (4). Since the integer variable in Problem (4) is binary, the B&B

algorithm can attain the globally optimal solution by iteratively searching a binary tree. Each node  $n$  in the tree is associated with a nonlinear sub-problem of Problem (4), where the constraints of the integer variables, i.e., constraint (3a), are modified. Specifically, some integer variables are determined while others are undetermined and relaxed into continuous variables within  $[0, 1]$ . By solving the corresponding nonlinear problem at node  $n$ , its local upper bound,  $b_U^n$ , can be obtained, because the feasible region of the corresponding nonlinear problem is larger than that of the original MINLP problem. The searching process of the B&B algorithm contains three iterative steps.

- Node selection: selecting a node from the unvisited node list of the tree.
- Evaluation: solving the corresponding nonlinear sub-problem of the node to obtain its local upper bound,  $b_U^n$ .
- Fathom decision: using the local upper bound,  $b_U^n$ , and the global lower bound,  $b_L$ , which is the optimal value of the objective function by far, to decide whether the node should be fathomed.

The searching process comes to an end by iteratively repeating the above three steps until the node list is empty.

In the following, we will first solve the corresponding nonlinear problem at each node, and then develop the complete B&B algorithm for Problem (4).

1) *Local Upper Bound Algorithm*: As mentioned before, at each node of the binary tree generated by the B&B algorithm, some channel allocation indicators are determined while others are relaxed into a continuous variable within  $[0, 1]$ . We use  $\rho^d$  to denote the matrix that stores the values of the determined channel allocation indicators. Then the nonlinear sub-problem of Problem (4) corresponding to node  $n(\rho^d)$  can be written as

$$\max_{\{p^D, \hat{\rho}\}} \min_{l \in \mathcal{L}} \hat{R}_l^D(p^D, \hat{\rho}, \rho^d), \quad (5)$$

subject to

$$\hat{\rho}_{kl} \in [0, 1], \quad \forall k \in \mathcal{K}, l \in \mathcal{L}, \quad (5a)$$

$$\sum_{l \in \mathcal{L}} \hat{\rho}_{kl} \leq 1, \quad \forall k \in \mathcal{K}, \quad (5b)$$

$$\hat{\rho}_{kl} = \rho_{kl}^d, \quad \forall \{k, l\} \in \mathcal{D}_n, \quad (5c)$$

$$\sum_{k \in \mathcal{K}} \hat{\rho}_{kl} p_{kl}^D \leq P_{\max}^D, \quad \forall l \in \mathcal{L}, \quad (5d)$$

$$p_{kl}^D \leq p_{kl}^{\max}, \quad \forall k \in \mathcal{K}, l \in \mathcal{L}, \quad (5e)$$

where  $\mathcal{D}_n$  is the index set of determined channel allocation indicators. It is obvious that the solution to Problem (5) serves as an upper bound of Problem (4).

We introduce a new variable  $s_{kl} = p_{kl}^D \hat{\rho}_{kl}$ . Then

$$p_{kl}^D = \begin{cases} 0, & \hat{\rho}_{kl} = 0, \\ s_{kl} / \hat{\rho}_{kl}, & \text{otherwise.} \end{cases} \quad (6)$$

Then Problem (5) can be modified into

$$\max_{\{\eta, s, \hat{\rho}\}} \eta(\rho^d), \quad (7)$$

TABLE I  
OPTIMAL B&B ALGORITHM

---

**Algorithm 1** Optimal B&B Algorithm

---

1: **initialization**

- Set node list:  $\mathcal{N} \leftarrow \{n(\emptyset)\}$ .
- Set global lower bound:  $b_L = -\infty$ .
- Set optimal solution:  $\rho^* \leftarrow \text{None}$ ,  $p^{D*} \leftarrow \text{None}$ .

2: **while**  $\mathcal{N} \neq \emptyset$  **do**

3:   **Node Selection**: pop the first node  $n(\rho^d)$  in  $\mathcal{N}$ .

4:   **Evaluation**: use interior-point method solve Problem (7) corresponding to  $n(\rho^d)$ .

5:   **Fathom Decision**:

6:   **if** Problem (7) corresponding to  $n(\rho^d)$  is infeasible **then**  
7:     Go to step 2.

8:   **else**

9:     Get  $\eta, \hat{\rho}, p^D$ . Set  $b_U^n = \eta$ .

10:    **if**  $\hat{\rho}$  is integral **then**

11:     An integer solution is found.

12:     **if**  $b_U^n > b_L$  **then**

13:       Update global lower bound and optimal solution.

14:        $b_L = b_U^n$ ,  $\rho^* = \hat{\rho}$ ,  $p^{D*} = p^D$ .

15:     **end if**

16:     Go to step 2.

17:   **else**

18:     **if**  $b_U^n < b_L$  **then**

19:       Prune node  $n(\rho^d)$ .

20:       Go to step 2.

21:     **else**

22:       Branch on node  $n(\rho^d)$ .

23:       **Variable Selection**: branch on the first undetermined variable in  $\hat{\rho}$ .

24:       Add the new nodes into  $\mathcal{N}$ .

25:       Go to step 2.

26:     **end if**

27:    **end if**

28:    **end if**

29: **end while**

---

subject to (5a), (5b), (5c), and

$$\sum_{k \in \mathcal{K}} s_{kl} \leq P_{\max}^D, \quad \forall l \in \mathcal{L}, \quad (7a)$$

$$s_{kl} \leq \hat{\rho}_{kl} p_{kl}^{\max}, \quad \forall k \in \mathcal{K}, l \in \mathcal{L}, \quad (7b)$$

$$\hat{R}_l^D(s, \hat{\rho}) \geq \eta, \quad \forall l \in \mathcal{L}, \quad (7c)$$

where

$$\hat{R}_l^D(s, \hat{\rho}) = \sum_{k \in \mathcal{K}} \log \left( 1 + \frac{\hat{\rho}_{kl} s_{kl}}{a_{kl} \hat{\rho}_{kl} + b_{kl} s_{kl}} \right). \quad (8)$$

Problem (7) is obviously equivalent to Problem (5). Furthermore, we can prove that (8) is a concave function [24] and Problem (7) is a convex optimization problem. Therefore, the interior-point method can be used to solve it and thus the local upper bound of each node can be obtained, which will be used to decide whether a node should be fathomed during the B&B searching process.

2) *B&B Algorithm*: After obtaining the local upper bound of each node, we can develop the B&B algorithm for Problem (4), which is summarized in Table I. For simplicity, we adopt the depth-first-search (DFS) [27] as the node selection rule and always choose the first undetermined channel indicator for variable selection process. However, the algorithm in Table I is

impractical due to its exponential computational complexity, i.e.,  $O(2^{KL})$ , which motivates us to accelerate it by ML techniques.

#### IV. ACCELERATING BY IMITATION LEARNING

In this section, we will use the ML technique, specifically the imitation learning method, to accelerate the algorithm in Table I. We will first discuss the optimal auxiliary prune policy and then give a brief introduction to the concept of imitation learning. Finally, we will introduce how to use it in our specific problem and illustrate the detailed training process.

##### A. Optimal Auxiliary Prune Policy

During the B&B searching process in Table I, there are two main goals: finding an optimal solution and guaranteeing its optimality by searching all feasible solutions and comparing with it. Most of the time is consumed in the latter one and a good prune policy can significantly reduce the computational complexity. The more nodes are pruned, the less time is consumed.

The original prune policy of the algorithm in Table I only includes three cases:

- The sub-problem is infeasible as shown in line 6 of Table I. If the relaxed nonlinear sub-problem corresponding to  $n(\rho^d)$  is infeasible, the related MINLP problem is also infeasible and then the node  $n(\rho^d)$  is fathomed.
- A feasible solution is found as shown in line 10 of Table I. If  $\hat{\rho}$  is an integral vector, the result is also a feasible solution of the related MINLP problem and then the node  $n(\rho^d)$  is fathomed.
- The local upper bound is smaller than the current global lower bound as shown in line 18 of Table I. If  $\hat{\rho}$  is not an integral vector and  $b_U^n < b_L$ , then the node  $n(\rho^d)$  would not lead to a better solution and is fathomed.

Besides the three cases mentioned above, all the non-optimal nodes should be pruned to accelerate the B&B algorithm. We can achieve this goal by using the method in [20], [21], where the aforementioned original prune policy in the B&B algorithm has been replaced with a learned policy that would prune all the non-optimal nodes. However, it is redundant to learn to prune the fathomed nodes since the original prune policy is compact and explicit. Therefore, we propose to simplify the learning task by keeping the original prune policy and learning an auxiliary prune policy to reinforce it. The auxiliary prune policy is supposed to prune the non-optimal nodes that are not fathomed instead of all the non-optimal ones. Our proposed simplification about learning task can reduce the complexity of target model. Generally, simpler model has smaller Vapnik-Chervonenkis (VC) dimension, which leads to fewer needed training samples and speeds up the training process consequently. In this way, the best auxiliary prune policy,  $\pi_p^*$ , should discard all non-optimal nodes that have not been fathomed. In the following, we introduce how to use imitation learning to learn the best auxiliary prune policy,  $\pi_p^*$ .

##### B. Basic Imitation Learning

Imitation learning has been widely used in sequential decision problems, where the learner tries to mimic an expert's action,

i.e., an oracle, to achieve the best performance in a supervised manner [28]. Generally, a sequential decision problem is defined by a state space,  $\mathcal{S}$ , an action space,  $\mathcal{A}$ , and a policy space,  $\Pi$ . A policy  $\pi \in \Pi$  reflects the mapping relations between the states and actions, i.e.,  $\pi(s) = a$ . The oracle,  $\pi^*$ , is a special policy that provides the optimal action,  $a^*$ , for any possible state  $s \in \mathcal{S}$ . The goal of imitation learning is to find a policy that mimics the oracle's actions.

We shall note that imitation learning is different from reinforcement learning. Although the goal of reinforcement learning is also to learn a policy, the best policy for reinforcement learning is unknown and should be learned by interacting with the environment in an unsupervised manner. Whereas, the best policy in imitation learning is known and can be learned in a supervised manner. Imitation learning better fits accelerating the B&B algorithm than reinforcement learning.

The B&B searching process in Table I can be formulated as a sequential decision-making process. At each node encountered during the searching process, we need to decide whether it is pruned using the prune policy and the decision can influence the whole B&B searching process. Here, the state space,  $\mathcal{S}$ , is the set of all the visited nodes with the corresponding global lower bound and the current optimal solution. The action space,  $\mathcal{A}$ , is  $\{\text{prune}, \text{branch}\}$  and the policy space is defined as  $\Pi_p$ .

Let  $\phi: \mathcal{S} \rightarrow \mathbb{R}^q$  be the feature mapping that finds a  $q$ -dimension feature-vector description for any state  $s \in \mathcal{S}$ . Given that our goal is to learn the best auxiliary prune policy,  $\pi_p^*$ , i.e., learn the optimal action,  $a^*$ , for any possible state  $s \in \mathcal{S}$ , the imitation learning problem can be converted into a supervised learning problem: the policy takes a feature-vector description of the state  $s$  as input and tries to predict the oracle action,  $a^*$ , as output. Moreover, because the action space,  $\mathcal{A}$ , is only two-dimensional, the imitation learning problem can be further converted into a binary classification problem.

##### C. Imitation Learning for Auxiliary Prune Policy

Now, we utilize the imitation learning method to learn the optimal auxiliary prune policy. As mentioned above, the imitation learning problem can be converted into a binary classification task with appropriate feature mapping,  $\phi$ . In this part, we will first discuss how to design appropriate feature mapping, and then introduce how to use SVM to train the binary classifier.

1) *Feature Design*: Feature design is very important for training classifiers. We need to dig out the features that are closely related to state  $s \in \mathcal{S}$ . In [20], problem-independent features have been used for general MILP problems, which can be extended to the MINLP problems. On the other hand, problem-dependent features have been exploited in [21] in addition to the problem-independent ones to further improve the performance of the classifier. Here, we adopt both kinds of features.

###### i. Problem-Independent Features

Problem-independent features focus on the structure of the binary tree generated by the B&B algorithm, which can be used in all MINLP problems regardless of applications. It consists of the following three categories [20].

- Node features: State  $s$  is highly relevant to current visiting node  $n$ , thus features computed from current node  $n$  are essential. They include the depth of node  $n$ , the plunge depth of node  $n$ , and the local upper bound,  $b_U^n$ , of node  $n$ .
- Branching feature: State  $s$  also depends on the branching variable leading to current node  $n$ , which is chosen at the variable selection step of the parent of node  $n$ . It is less important than node features in the auxiliary prune policy learning problem [20]. Here we use the value of the branching variable as the only branching feature.
- Tree features: Features computed from the binary tree are also important for describing state  $s$ , which includes current global lower bound,  $b_L$ , and the number of solutions obtained so far.

Most of the features mentioned above vary largely for different problems. For example, the objective values of problems with different CSI vary a lot. On the other hand, the plunge depths of nodes vary with problem sizes. Therefore, we normalize the features involving bound with the local upper bound at the root node as well as those involving depth with the maximum depth of the binary tree. This normalization process makes the above features size-independent, i.e., independent on the numbers of CUs and D2D pairs, which can help improve the generalization ability of the proposed learned prune policy.

#### ii. Problem-Dependent Features:

Problem-dependent features, as the name implies, are closely related to the specific problem we solve. To choose appropriate problem-dependent features, we need to check the original Problem (3) and find the key parameters related to the channel and power allocation for D2D communications. It is obvious that CSI and power constraints are two key factors. Therefore, we try to design problem-dependent features according to these two factors in the following.

- CSI feature: At node  $n$ , there are four different kinds of CSI involved, i.e.,  $h_{kl}^{CD}$ ,  $h_k^{CB}$ ,  $h_l^D$ , and  $h_l^{DB}$ , when the first undetermined channel allocation indicator is  $\hat{\rho}_{kl}$ . In Proposition 2, we have introduced two new variables  $a_{kl}$  and  $b_{kl}$  that combine these four kinds of CSI with the data rate of D2D pair  $l$  on the channel of CU  $k$ , i.e.,  $R_{kl}^D$ . According to (9), we can formulate the CSI feature as  $\log(1 + 1/(a_{kl} + b_{kl}))$ . On the other hand, both  $a_{kl}$  and  $b_{kl}$  are related to  $R_{\min}^C$  that varies across different problems. Therefore, we normalize the CSI feature with  $R_{\min}^C$  and get the size-independent CSI feature as  $f(CSI) = (\log(1 + 1/(a_{kl} + b_{kl}))) / R_{\min}^C$ .
- Power feature: At node  $n$ , when the first undetermined channel allocation indicator is  $\hat{\rho}_{kl}$ , we formulate a function of  $p_{kl}$  as the power feature. According to (4a), we can denote the power feature as  $p_{kl}^{\max}$ . Because  $p_{kl}^{\max}$  also varies across different problems, the normalized power feature can be similarly formulated as  $g(p_{kl}) = K L p_{kl}^{\max} / \sum_{k=1}^K \sum_{l=1}^L p_{kl}^{\max}$ .

These two aforementioned problem-dependent features are also size-independent. Combined with the size-independent problem-independent features, we can expect that the auxiliary prune policy we learn from the oracle of specific scenarios can be applied to scenarios with different sizes, i.e., different numbers

of CUs and D2D pairs, which will be confirmed by the test results in Section V.

2) *Binary Classifier Learning*: After choosing an appropriate feature mapping,  $\phi$ , we can handle the binary classification problem. We use SVM to train the classifier, which is widely used for classification. The SVM aims to find a mapping of training examples so that they can be divided by a clear gap that is as wide as possible in the new space [29]. We stack the eight features mentioned above in an 8-dimension vector as the input and use the corresponding action,  $a^*$ , as the output. Moreover, we denote *branch*, i.e., optimal node, as 1, and *prune*, i.e., non-optimal node, as 0.

During the B&B searching process, mistakes made at early stages are more serious. For example, the root node is always supposed to be classified as branching. If it is misclassified as pruning, the searching process ends and no feasible solution can be found. On the other hand, it is more serious to prune the optimal nodes by mistake than to keep non-optimal nodes. If an optimal node is misclassified as non-optimal and is pruned, the optimal solution cannot be found either. However, if a non-optimal node is misclassified as optimal, just more time is consumed but the optimality of result is still preserved.

To deal with the first problem mentioned above, we place higher weights on the training examples from the nodes with small depth. Because the subtree size decreases exponentially with the node's depth, the weight related to node's depth,  $\omega_1$ , is also supposed to decrease exponentially for nodes at different depths. Therefore, we set  $\omega_1$  for a node with depth  $d$  as  $\omega_1 = A e^{-\frac{Bd}{D}}$ , where  $D$  is the maximum depth of the B&B tree,  $A$  and  $B$  are the parameters to be tuned during the training process. Specifically, we set  $A = 5$  and  $B = 2.68$  according to [20], which enables the weight of the root node to be 5 and the weights for nodes at different depths decrease at the rate  $2.68/D$ . Nevertheless, different values of  $A$  and  $B$  will impose different impacts on the results. Increasing  $A$  will reinforce the optimality by placing higher weights while increasing  $B$  will increase the decay rate and differentiate more clearly for nodes at different depths. In practice, we can choose appropriate  $A$  and  $B$  according to different goals.

As for the second problem mentioned above, we place higher weights on the training examples from optimal nodes and denote the weight related to node's optimality as  $\omega_2$ . We set  $\omega_2$  as 1 for non-optimal nodes and tune it for optimal ones from  $\{1, 2, 4, 8\}$  during the training process. There are two main benefits of placing  $\omega_2$ : avoiding the serious optimal node misclassification issue mentioned above and dealing with the dataset imbalance issue. Dataset imbalance issue means the number of the optimal nodes is much smaller than that of the non-optimal ones, which can be mitigated by placing higher weights at the optimal nodes. Combined these two parts mentioned above, the total weight,  $\omega$ , for each training sample is the product of  $\omega_1$  and  $\omega_2$ , i.e.,  $\omega = \omega_1 \times \omega_2$ .

#### D. Iterative Training Process

The binary classifier is trained in the supervised manner. The general method puts all the nodes searched by the

TABLE II  
TRAINING PROCESS FOR IMITATION LEARNING WITH DAGGER

**Algorithm 2** Training Process for Imitation Learning with Dagger

---

```

1: initialization
   • Set policy:  $\pi_p^{(1)} = \pi_p^*$ .
   • Set training dataset:  $\mathcal{T} = \emptyset$ .
2: for  $m = 1$  to  $M$  do
3:   for problem  $Q$  in  $\mathcal{Q}$  do
4:      $\mathcal{T}^{(Q)} \leftarrow \text{DataCollection}(Q, \pi_p^{(m)})$ 
5:      $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{T}^{(Q)}$ 
6:   end for
7:    $\pi_p^{(m+1)} \leftarrow \text{train SVM (classifier) using } \mathcal{T}$ .
8: end for
9: return Best  $\pi_p^{(m)}$  on the validation set.

```

---

TABLE III  
DATA COLLECTION ALGORITHM: DATA COLLECTION( $Q, \pi_p$ )

**Algorithm 3** DataCollection( $Q, \pi_p$ )

---

```

1: initialization
   • Set node list:  $\mathcal{N}_Q \leftarrow \{n(\emptyset)\}$ .
   • Set dataset:  $\mathcal{D} = \emptyset$ .
2: while  $\mathcal{N}_Q \neq \emptyset$  do
3:   Node Selection: pop the first node  $n_Q(\rho^d)$  in  $\mathcal{N}_Q$ .
4:   Evaluation: use interior-point method solve Problem (7)
     corresponding to  $n_Q(\rho^d)$ .
5:   Fathom Decision:
6:   if Node  $n_Q(\rho^d)$  is fathomed by original B&B then
7:     Go to step 2.
8:   else
9:      $\mathcal{D} \leftarrow \{f(n_Q(\rho^d)), \pi_p^*(n_Q(\rho^d))\}$ 
10:    if  $\pi_p(n_Q(\rho^d)) = \text{branch}$  then
11:      Branch on node  $n_Q(\rho^d)$ .
12:    Variable Selection: branch on the first undetermined
      variable in  $\hat{\rho}$ .
13:    Add the new nodes into  $\mathcal{N}_Q$ .
14:  end if
15:  Go to step 2.
16: end if
17: end while
18: return  $\mathcal{D}$ 

```

---

algorithm in Table I into the training set. However, this method is memory-consuming because the number of nodes increases exponentially. Therefore, we adopt an iterative training algorithm, dataset aggregation (Dagger) [20], [30], to achieve more efficient training.

Dagger is widely used in the imitation learning to provide a learning reduction with strong performance guarantee [30]. It proceeds by collecting a dataset at each iteration using current policy and then trains a new policy with the aggregation of all collected datasets. The iterative dataset collection process can enhance the generalization ability of the model to some extent.

The detailed training procedures for imitation learning using Dagger are shown in Tables II and III. At the first iteration, we use the oracle,  $\pi_p^*$ , as the initial policy and denoted as  $\pi_p^{(1)}$ . We search each problem in the problem set,  $\mathcal{Q}$ , with  $\pi_p^{(1)}$  and collect data into the training dataset,  $\mathcal{T}$ . Then we use  $\mathcal{T}$  to train SVM and get a new learned policy denoted as  $\pi_p^{(2)}$ . We repeat

this process for  $M$  rounds and choose the policy,  $\pi_p^{(m)}$ , that performs the best on the validation set.

Note that, some steps in Algorithm 3 are similar with those in Algorithm 1 because the B&B tree needs to be searched in both algorithms. However, Algorithm 3 is mainly developed to collect the training data for the iterative training process in Table II. The new learned policy has been adopted in the Algorithm 3 for pruning decision, which makes it different from that in Algorithm 1.

Here, we make some improvements on the training process to accelerate it and also deal with the dataset imbalance problem. Specifically, we only collect the nodes that are visited under current policy and are not fathomed by the original B&B prune policy during the data collection process shown in Table III. It is different from the data collection process in [20] and [21], which includes all the visiting nodes into the dataset. There exist two main reasons to make this improvement.

- When visiting nodes that are fathomed by the original B&B prune policy, the learned auxiliary prune policy is not needed. Therefore, ignoring those fathomed nodes would not induce loss in performance but can speed up the training process of imitation learning. Note that if we discard the original prune policy and learn a new one to replace it as proposed in [20], [21], the fathomed nodes cannot be neglected during the data collection process. Therefore, the improvement on the training process is directly led by keeping the original prune policy.
- The nodes that are fathomed by the original B&B prune policy are non-optimal. Therefore, discarding these nodes from the dataset can decrease the number of non-optimal nodes and mitigate the dataset imbalance issue mentioned above to some extent.

## V. PERFORMANCE TEST RESULTS

In this section, we will test the performance of the proposed imitation learning method on accelerating the algorithm in Table I. All the codes are implemented in python 3.6 except the interior-point algorithm that is implemented in Matlab. They are implemented on a computer station with one 4-core Intel processor and 128 GB of memory. To avoid over-fitting, we set  $M = 4$  for all the imitation learning training process and use LIBSVM [31] for the step of training SVM (classifier) in the following. Given that the main goal of this paper is using a ML technique to accelerate the B&B algorithm, we mainly test the performance of the ML technique with specific focus on scalability and generalizability. Moreover, to honor the tradition in the ML community, all the testing results are presented in tables with specific numerical values.

### A. System Setup

We consider a single-cell network with a radius of 500 m as in Fig. 1. The eNB is located in the center of the cell and the CUs are distributed uniformly in the cell. According to [18], the transmitter of each D2D pair is also distributed uniformly in the cell and the corresponding receiver is distributed in a

TABLE IV  
SIMULATION PARAMETERS

| Parameter                                           | Value                          |
|-----------------------------------------------------|--------------------------------|
| Cell radius                                         | 500 m                          |
| D2D distance, $r_{\min}$ , $r_{\max}$               | 15 m, 50 m                     |
| Noise spectral density                              | -174 dBm/Hz                    |
| Path loss model for cellular links                  | $128.1+37.6\log(d[\text{km}])$ |
| Path loss model for cellular D2D links              | $148+40\log(d[\text{km}])$     |
| Shadowing standard deviation                        | 10 dB                          |
| Maximum transmitter power of CU, $P_{\max}^C$       | 20 dBm                         |
| Maximum transmitter power of D2D pair, $P_{\max}^D$ | 20 dBm                         |
| Minimum data rate of CU, $R_{\min}^C$               | 2 bit/s/Hz                     |

disk centered by the transmitter with fixed uniform link distance distribution between  $r_{\min}$  and  $r_{\max}$ . Our simulation parameters are summarized in Table IV.

### B. Influence of the Number of Training Samples

An important test of the proposed method is how many training samples are enough to learn a good auxiliary prune policy since training samples are always difficult and expensive to get, especially in wireless networks where a large number of real data are hard to obtain.

We test the performance of the proposed method with different numbers of training samples for scenarios with different numbers of CUs and D2D pairs. Because we usually have a small labeled dataset for wireless problems, we set the ratio between the number of training samples and that of testing samples as 10:1 for the subsequent test and use 20 testing samples for each scenario. During the testing stage, the learned model will be applied to every encountered node while searching the B&B tree for each testing sample. Given that the number of nodes of the B&B tree is exponentially increasing, 20 testing samples include hundreds even thousands of testing nodes, which are sufficient for performance evaluation.

The testing results are shown in Table V. We use four metrics to measure the performance of the proposed method. *Ogap* is the optimality gap, which means the gap between the optimal objective function and the one achieved by the proposed imitation learning method. *Speed* refers to the speedup with respect to the original B&B searching process, which is computed as the ratio between the number of nodes explored without and with the learned auxiliary policy. Note that all the running time is tested with the same hardware set. *Optimal recognition rate* is the accuracy rate of the learned policy on recognizing optimal nodes, which is positively related to *ogap*. *Extra prune rate* is the percentage of the non-optimal nodes that are not fathomed by the original B&B prune policy but can be recognized by the learned auxiliary policy. It is obvious that *speed* and *extra prune rate* are also positively correlated. Note that the values of *ogap* and *speed* are the average of the 20 testing samples while those of *optimal recognition policy* and *extra prune policy* are the average of the

TABLE V  
PERFORMANCE OF IMITATION LEARNING WITH DIFFERENT NUMBERS OF TRAINING SAMPLES(a)  $K = 5$ ,  $L = 2$ 

| Number of training samples | 50     | 100    | 150    | 200    |
|----------------------------|--------|--------|--------|--------|
| Ogap                       | 3.88%  | 3.23%  | 2.27%  | 2.01%  |
| Speed                      | 2.50x  | 2.21x  | 2.17x  | 2.06x  |
| Optimal recognition rate   | 89.22% | 93.27% | 93.33% | 93.38% |
| Extra prune rate           | 30.88% | 30.32% | 29.00% | 28.20% |

(b)  $K = 7$ ,  $L = 2$ 

| Number of training samples | 50     | 100    | 150    | 200    |
|----------------------------|--------|--------|--------|--------|
| Ogap                       | 7.81%  | 7.62%  | 7.06%  | 6.96%  |
| Speed                      | 9.51x  | 4.91x  | 4.89x  | 4.82x  |
| Optimal recognition rate   | 80.46% | 86.92% | 88.65% | 90.21% |
| Extra prune rate           | 18.07% | 38.24% | 41.11% | 40.96% |

(c)  $K = 5$ ,  $L = 3$ 

| Number of training samples | 50     | 100    | 150    | 200    |
|----------------------------|--------|--------|--------|--------|
| Ogap                       | 14.54% | 13.08% | 12.54% | 11.92% |
| Speed                      | 15.79x | 15.45x | 13.91x | 11.66x |
| Optimal recognition rate   | 83.33% | 85.46% | 86.21% | 86.89% |
| Extra prune rate           | 42.22% | 48.47% | 50.00% | 50.91% |

hundreds and even thousands of testing nodes of the B&B trees for the 20 testing samples.

Our algorithm is expected to attain a small objective function gap while reducing the computational complexity as much as possible. As shown in Table V, the proposed method can speed up the B&B searching process by 2.06 times with only 2.01% loss of accuracy for the scenario with  $K = 5$ ,  $L = 2$ , and can speed up the B&B searching process by 4.82 times with 6.96% loss of accuracy for the scenario with  $K = 7$ ,  $L = 2$ , both with only 200 training samples. For the more complicated scenario with  $K = 5$ ,  $L = 3$ , the proposed method can speed up the B&B searching process by 11.66 times with 11.92% loss of accuracy while using 200 training samples. If reducing to 50 training samples, the proposed method can still achieve low performance gaps of 3.88%, 7.81% and 14.54% for the three scenarios, respectively. Comparing to the one million training samples used in [13], the proposed imitation learning method can achieve a good performance with a small set of training samples, and thus is more feasible for wireless networks. On the other hand, *ogap* and *speed* both decrease with the increase of the number of training samples. As we can imagine, using more training samples will increase the optimality but will also induce more computational complexity, which indicates the optimality and computational complexity trade-off issue. Small training sets lead to a policy with lower computational complexity but also lower optimality, while larger training sets lead to the opposite results. This can help us choose appropriate numbers of training samples in practice according to our specific goals instead of using as much training samples as possible. Furthermore, with the increase on the number of training samples, the performance for the scenario with  $K = 5$ ,  $L = 2$  improves very slowly while

TABLE VI

PERFORMANCE OF IMITATION LEARNING WITH DIFFERENT SETS OF FEATURES FOR THE SCENARIO WITH  $K = 5$ ,  $L = 2$

| Feature set              | Problem-independent features only | Two kinds of features combined |
|--------------------------|-----------------------------------|--------------------------------|
| Ogap                     | 3.11%                             | 2.01%                          |
| Speed                    | 1.93x                             | 2.06x                          |
| Optimal recognition rate | 91.45%                            | 93.38%                         |
| Extra prune rate         | 26.48%                            | 28.20%                         |

those for the scenarios with  $K = 7$ ,  $L = 2$  and  $K = 5$ ,  $L = 3$  improve at a stable speed. And the achieved optimality for the scenario with  $K = 5$ ,  $L = 2$  is always higher than those for the scenarios with  $K = 7$ ,  $L = 2$  and  $K = 5$ ,  $L = 3$  when using the same number of training samples. These results indicate that the policy for the scenario with  $K = 5$ ,  $L = 2$  is fully trained but those for the scenarios with  $K = 7$ ,  $L = 2$  and  $K = 5$ ,  $L = 3$  are not yet. Larger training sets are needed for larger scale problems than for smaller scale problems to achieve the same optimality.

### C. Influence of Feature Selection

Feature selection is very important for training classifier and we have used eight problem-independent and problem-dependent features. As mentioned in Section IV-C, problem-independent features are general for all MINLP problems while problem-dependent ones need to be carefully selected according to specific applications. We will test the importance of both kinds of features in the following to obtain some insights about feature selection in practice.

We test on the scenario with  $K = 5$ ,  $L = 2$  using 200 training samples and 20 testing samples. The results of training with problem-independent features only and with both kinds of features are summarized in Table VI. From the table, the proposed method can speed up the B&B searching process by 1.93 times with only 3.11% performance loss with only problem-independent features while adding the problem-dependent features can improve the optimality and reduce computational complexity simultaneously. This result suggests the necessity of the problem-dependent features in reinforcing the performance.

Furthermore, we rank all the eight features using *F-test*, i.e., joint hypotheses test, which is widely applied in feature selection for ML. We find that the most important two features are the local upper bound,  $b_U^n$ , and the global lower bound,  $b_L$ . The learned policy is inclined to branch on the nodes whose difference between  $b_U^n$  and  $b_L$  are very small, which means these nodes have very large probabilities to be the optimal ones. The less two important features are two problem-dependent features, which coincides with the fact mentioned above that the proposed method can also achieve a satisfactory performance without problem-dependent features.

Inspired from the above observation, we can implement the imitation learning method in a two-step paradigm in practice. First, we only use problem-independent features for training. If the optimality and the computational complexity of the learned policy is acceptable, then we can skip the intricate design

TABLE VII

CROSS GENERALIZATION TEST ON THE PERFORMANCE OF IMITATION LEARNING FOR SCENARIOS WITH  $K = 5$ ,  $L = 2$  AND  $K = 7$ ,  $L = 2$

| Cross generalization test combination | Policy(5,2) on Problem(5,2) | Policy(5,2) on Problem(7,2) | Policy(7,2) on Problem(5,2) | Policy(7,2) on Problem(7,2) |
|---------------------------------------|-----------------------------|-----------------------------|-----------------------------|-----------------------------|
| Ogap                                  | 2.01%                       | 3.70%                       | 9.44%                       | 6.96%                       |
| Speed                                 | 2.06x                       | 3.46x                       | 3.25x                       | 4.82x                       |
| Optimal recognition rate              | 93.38%                      | 90.65%                      | 85.29%                      | 90.21%                      |
| Extra prune rate                      | 28.20%                      | 24.77%                      | 47.35%                      | 40.96%                      |

process of problem-dependent features. Otherwise, we then add problem-dependent features and train a new learned policy by using both kinds of features.

### D. Generalization to Scenarios With Different Sizes

Generalization ability is another important property of ML techniques. As mentioned in Section IV-C, we expect our method can be generalized to scenarios with different problem sizes. In the following, we first test whether the proposed imitation learning method has generalization ability, which includes generalization abilities to both the larger and the smaller scenarios. Then we test how strong its generalization ability is.

First, we do cross generalization tests on scenarios with  $K = 5$ ,  $L = 2$  and  $K = 7$ ,  $L = 2$ . We use 200 training samples and 20 testing samples for both scenarios. The results are summarized in Table VII, where *Policy*( $k, l$ ) means the policy learned from the scenario with  $K = k$ ,  $L = l$ , and *Problem*( $k, l$ ) means the problem on the scenario with  $K = k$ ,  $L = l$ . It is obvious that the proposed method has the generalization ability to the larger and the smaller scale problems. Specifically, the optimality by using *Policy*(5, 2) on *Problem*(7, 2) is even higher than using *Policy*(7, 2) on *Problem*(7, 2). This result seems not intuitive but is reasonable. As mentioned above, *Policy*(7, 2) is not fully trained and more training samples are needed to improve its performance. Meanwhile, *Policy*(5, 2) is fully trained and has a strong generalization ability. Therefore, *Policy*(5, 2) can achieve better performance than *Policy*(7, 2) on *Problem*(7, 2). This result suggests that if there are not enough training samples for large scenario, using a fully trained policy from a small scenario can achieve better performance than training a policy with insufficient samples from the large scenario.

After checking the existence of the generalization ability of the proposed method, we now turn to test how strong its generalization ability is, especially the generalization ability to the larger scenarios. Given that *Policy*(5, 2) is fully trained and will improve very little with more training samples, we regard *Policy*(5, 2) learned with 200 training samples as the best policy. We test the performance of this policy on scenarios with different problem sizes. For each scenario, we use 20 testing samples. The result is shown in Table VIII, where problem size ( $k, l$ ) denotes the scenario with  $K = k$ ,  $L = l$ . Note that, the largest scenario we consider here includes 10 CUs and 2 D2D pairs, because the training samples for larger-scale scenarios are extremely hard to obtain due to the exponentially increasing complexity of the B&B algorithm.

TABLE VIII  
PERFORMANCE OF IMITATION LEARNING FOR SCENARIOS WITH DIFFERENT SIZES BY USING POLICY(5, 2)

| Problem size             | (7,2)  | (5,3)  | (8,2)  | (10,2) |
|--------------------------|--------|--------|--------|--------|
| Ogap                     | 3.70%  | 11.53% | 5.46%  | 7.69%  |
| Speed                    | 3.46x  | 4.87x  | 7.25x  | 22.13x |
| Optimal recognition rate | 90.65% | 90.47% | 89.78% | 84.91% |
| Extra prune rate         | 24.77% | 31.36% | 26.87% | 28.42% |

From Table VIII, the performance of *Policy*(5, 2) still remains acceptable even for *Problem*(10, 2) whose complexity is  $2^{10}$  larger than that of *Problem*(5, 2). The above result shows that we can use the policy learned from the smaller scale problems to the larger scale problems with good performance. On the other hand, the *optimal recognition rate* decreases with the increase of the problem scale. This result implies that the generalization ability is constrained by the problem size of training samples and deteriorates with the size of objective problems. This phenomenon inspires us to use mixed training strategy and soft-decision algorithm to further improve the training performance in the next section.

## VI. DISCUSSION AND FURTHER IMPROVEMENT

Inspired by the test results in Section V, we come up with two important questions: how to strengthen the generalization ability of the proposed imitation learning method, and how to dynamically control the optimality and computational complexity trade-off. In this section, we will focus on these two questions and come up with some methods to solve them.

### A. Mixed Training Strategy

As shown in Section V-D, the generalization ability is good but also limited. Policy learned from small-scale problems can be generalized to large-scale problems, but the performance deteriorates with the increase of the problem scale. It is obvious that the generalization ability is constrained by the problem scale of training samples.

In [21], the transfer learning method has been used to solve this problem. Its basic idea is adding few unlabeled training samples from large-scale problems and making use of an exploration policy to enhance the generalization ability. However, the exploration policy, which is crucial for transfer learning, is always hard to design for the MINLP problems in wireless networks. Furthermore, the transfer learning method would induce higher computational complexity for training process. Therefore, we propose to use the mixed training strategy to overcome the aforementioned shortcomings.

Specifically, if we want to solve *Problem*( $k, l$ ), we use a training set consists of many samples from the problems of smaller size than *Problem*( $k, l$ ) and very few samples from *Problem*( $k, l$ ). Different from [21], samples from *Problem*( $k, l$ ) are also labeled. This will not cost much overhead because it is not difficult to get very few labeled training samples from the large-scale problems in practice. For instance, we can make use of system history record to get some optimal allocation results

TABLE IX  
PERFORMANCE OF IMITATION LEARNING FOR SCENARIOS WITH DIFFERENT SIZES BY USING MIXED TRAINING STRATEGY

| Problem size             | (7,2)  | (5,3)  | (8,2)  | (10,2) |
|--------------------------|--------|--------|--------|--------|
| Ogap                     | 2.91%  | 10.59% | 2.64%  | 2.09%  |
| Speed                    | 3.39x  | 6.74x  | 7.90x  | 2.46x  |
| Optimal recognition rate | 91.55% | 91.55% | 92.50% | 95.35% |
| Extra prune rate         | 24.09% | 42.45% | 31.58% | 3.9%   |

and use them as training samples. Since the proposed algorithm needs very few training samples, the cost of gathering historical data would be fairly low. In this way, we reinforce the generalization ability without designing complicated exploration policy and consuming more training time. The only overhead is getting few labeled training samples from the large-scale problems, which is acceptable.

We implement the mixed training strategy for each scenario with the sizes in Table VIII to test the effectiveness of the proposed strategy. For each scenario, we use a mixed training set consisting of 190 samples from the scenario with  $K = 5$ ,  $L = 2$  and 10 samples from itself. Moreover, we use 20 testing samples for each scenario. The results are summarized in Table IX. Comparing Tables VIII and IX, *ogap* achieved by the mixed training strategy is always lower than that achieved by using the original training strategy. For some specific scenarios, it can even achieve lower computational complexity at the same time. The results suggest that the mixed training strategy achieves better optimality than using small-scale problems' training samples only. This method can achieve a trade-off between the learned policy's performance and the cost of getting training samples from the large-scale problems.

### B. Soft-Decision Algorithm

We have thus far used SVM to train the classifier to learn the optimal prune policy. However, it cannot achieve dynamic control over optimality and computational complexity. Because the result of SVM classifier is either 0 or 1. Once the classifier is decided, the category of each node is determined and we cannot modify any more. If we want to get a less optimal solution with higher speed, we have to turn to a new policy, which is not convenient in practical systems.

To deal with this problem, we adopt the forward neural network (FNN) to replace SVM as the classifier inspired by [21], [32]. For the binary classification problem, FNN can output the probability of each class rather than the definite result. It provides us with a soft-decision result and we can use it to dynamically control the optimality and computational complexity.

Specifically, we construct a  $J$ -layer FNN. We use Relu, i.e.,  $\text{Relu}(x) = \max(0, x)$ , as the activation function for  $J - 1$  hidden layers and use softmax for the output layer to indicate the probability of each class. The label  $\mathbf{y}$  of the optimal and non-optimal nodes are denoted as (1,0) and (0,1), respectively. And the loss function is the weighted cross-entropy, as

$$\text{Loss} = -\omega[1]\mathbf{y}[1]\log(\mathbf{o}[1]) - \omega[2]\mathbf{y}[2]\log(\mathbf{o}[2]),$$

TABLE X  
DYNAMIC SOFT-DECISION ALGORITHM

**Algorithm 4** Dynamic Soft-Decision Algorithm

```

1: initialization
   • Set node list:  $\mathcal{N}_Q \leftarrow \{n(\emptyset)\}$ .
   • Set threshold:  $\tau = 0.5$ .
2: while Speed or Ogap is not achieved do
3:   while  $\mathcal{N}_Q \neq \emptyset$  do
4:     Node Selection: pop the first node  $n_Q(\rho^d)$  in  $\mathcal{N}_Q$ .
5:     Evaluation: use interior-point method solve Problem (7) corresponding to  $n_Q(\rho^d)$ .
6:     Fathom Decision:
7:     if Node  $n_Q(\rho^d)$  is fathomed by original B&B then
8:       Go to step 2.
9:     else
10:      if  $\pi_p(n_Q(\rho^d))[1] \geq \tau$  then
11:        Branch on node  $n_Q(\rho^d)$ .
12:      Variable Selection: branch on the first undetermined variable in  $\hat{\rho}$ .
13:      Add the new nodes into  $\mathcal{N}_Q$ .
14:    end if
15:    Go to step 2.
16:  end if
17: end while
18: Increase or Decrease  $\tau$ .
19: end while

```

where  $\mathbf{o}$  is the output vector of the FNN and  $\omega$  is the weight vector for each class.  $\omega$  is closely related to  $\omega_1$  and  $\omega_2$  mentioned in Section IV-C. Specifically,  $\omega[1] = \omega_1 \times \omega_2$  and  $\omega[2] = \omega_1$ . This loss function is different from the one in [21] that does not take the depth-dependent weights,  $\omega_1$ , into consideration.

With the help of DAgger, FNN can be trained in a similar way to SVM classifier suggested in Tables II and III. After FNN-based policy is learned, we can get the output probability vector,  $\mathbf{o}_n$ , for node  $n$ , during the B&B searching process. Then we compare the optimal probability  $\mathbf{o}_n[1]$  with a given threshold,  $\tau$ . If  $\mathbf{o}_n[1] > \tau$ , node  $n$  is classified as the optimal node and is to be branched on; otherwise, node  $n$  is classified as the non-optimal node and is to be pruned. The threshold,  $\tau$ , can be used to control the optimality and computational complexity trade-off. For example, if we want to get a less optimal solution with higher speed, we do not need to train a new policy. We only need to reduce  $\tau$  iteratively to satisfy our goal. This dynamic control process is summarized in Table X.

We use the scenario with  $K = 5$ ,  $L = 2$  as an example to compare the performance of the SVM-based and FNN-based imitation learning methods. We use 200 training samples and 20 testing samples for each method. In order to make a fair comparison with the loss function used in [21], we adopt the FNN structure in [21] and construct an FNN with three hidden layers with 16, 32, and 16 nodes, respectively. We set the number of training epochs to be 30 and the batch size to be 128. We set the *ogap* limit of the FNN-based method as the *ogap* that can be attained by the SVM-based method, i.e., 2.01%. We change  $\tau$  by 0.01 for each iteration. The comparison results

TABLE XI  
PERFORMANCE OF SVM-BASED AND FNN-BASED IMITATION LEARNING FOR THE SCENARIO WITH  $K = 5$ ,  $L = 2$

| Method                                                  | Speed |
|---------------------------------------------------------|-------|
| SVM-based imitation learning                            | 2.06x |
| FNN-based imitation learning with new loss function     | 1.74x |
| FNN-based imitation learning with loss function in [21] | 1.69x |

are summarized in Table XI. Because the *optimal recognition rate* and *extra prune rate* change iteratively with  $\tau$  in the FNN-based method, we only focus on *speed* here to compare the SVM-based and FNN-based methods in a fair way. From Table XI, the computational complexity of our proposed method is slightly lower than that in [21]. It also suggests that the speed achieved by the FNN-based method is lower than that of the SVM-based method while achieving the same optimality. Because the FNN-based method needs iteration process to tune appropriate threshold  $\tau$  for each problem, it will consume more time than the SVM-based method to achieve dynamic control.

## VII. CONCLUSIONS AND FUTURE RESEARCH

This paper introduces a hybrid ML and mathematical method to solve resource allocation problems in D2D communication networks, which are usually formulated as MINLP problems. The main goal of leveraging the ML technique is to overcome the disadvantages of traditional mathematical techniques and get an efficient algorithm for resource allocation in D2D communications. The key idea is to accelerate the B&B algorithm, the widely-used globally optimal algorithm for MINLP problems, by learning a good auxiliary prune policy. This is achieved by the imitation learning method. Extensive experiment results have demonstrated that the proposed method can achieve good optimality and reduce computational complexity at the same time with only hundreds of training samples. It also has the generalization ability to large-scale problems. These two aforementioned properties are preferred in wireless networks. To further improve the generalization ability of the proposed method, we have also developed the mixed training strategy, which can balance the learned policy's performance and the cost of getting training samples from large-scale problems. Moreover, we have utilized a DNN as the binary classifier and proposed a novel loss function to dynamically control the optimality and computational complexity. Our studies in this work can be also applied to the MINLP problems in other wireless communication systems, such as communication mode selection and multi-cell resource allocation problems in D2D networks [23], [25], [26].

There still exist some problems about accelerating the B&B algorithm for resource allocation problems by imitation learning method. First, the optimality of the proposed method is guaranteed by the optimality of the B&B algorithm. However, optimal solutions can even not be achieved by the B&B algorithm for more difficult problems whose corresponding nonlinear problems are non-convex. Our proposed method can still be used for the non-convex problems to achieve a faster B&B algorithm, but the optimality cannot be guaranteed since the B&B algorithm is not optimal in that case. Second, our proposed method can speed up the B&B algorithm, but sometimes the speed up rates are not

high enough as the test results suggest. Attempting to further speed up the B&B algorithm while remaining the optimality at the same time is an important future direction. Also, finding other effective method to avoid dataset imbalance problem, further reducing the number of training samples, and even adopting new ML techniques to accelerate resource allocation are very interesting issues for fine-tuning our proposal.

#### APPENDIX A PROOF OF PROPOSITION 1

From [24], if D2D pair  $l$  is allowed to reuse the channel of CU  $k$ , the following equation should be satisfied to maximize the minimum data rate of all D2D pairs and guarantee the data rate of CU  $k$

$$\log \left( 1 + \frac{p_k^C h_k^{CB}}{\sigma_N^2 + p_{kl}^D h_l^{DB}} \right) = R_{\min}^C.$$

Then, we have

$$p_k^C = (2^{R_{\min}^C} - 1) \frac{\sigma_N^2 + p_{kl}^D h_l^{DB}}{h_k^{CB}}.$$

From (3c) and (3e), we can get the new bound of  $p_{kl}^D$  as  $p_{kl}^D \leq p_{kl}^{\max}$ , where

$$p_{kl}^{\max} = \min \left\{ P_{\max}^D, (1/h_l^{DB})(p_{\max}^C h_k^{CB} / (2^{R_{\min}^C} - 1) - \sigma_N^2) \right\}.$$

#### APPENDIX B PROOF OF PROPOSITION 2

From Proposition 1, the data rate of D2D pair  $l$  on the channel of CU  $k$  can be rewritten as

$$\begin{aligned} R_{kl}^D(p^C, p^D, \rho) &= \log \left( 1 + \frac{\rho_{kl} p_{kl}^D h_l^D}{\sigma_N^2 + p_k^C h_{kl}^{CD}} \right) \\ &= \log \left( 1 + \frac{\rho_{kl} p_{kl}^D h_l^D}{\sigma_N^2 + \frac{(2^{R_{\min}^C} - 1) h_{kl}^{CD} \sigma_N^2}{h_k^{CB}} (\sigma_N^2 + p_{kl}^D h_l^{DB})} \right) \\ &\triangleq \hat{R}_{kl}^D(p^D, \rho). \end{aligned}$$

For convenience, we introduce

$$\begin{aligned} a_{kl} &\triangleq \frac{\sigma_N^2}{h_l^D} + \frac{(2^{R_{\min}^C} - 1) h_{kl}^{CD} \sigma_N^2}{h_l^D h_k^{CB}}, \\ b_{kl} &\triangleq \frac{(2^{R_{\min}^C} - 1) h_{kl}^{CD} h_l^{DB}}{h_l^D h_k^{CB}}. \end{aligned}$$

Then,  $\hat{R}_{kl}^D(p^D, \rho)$  can be rewritten as

$$\hat{R}_{kl}^D(p^D, \rho) = \log \left( 1 + \frac{\rho_{kl} p_{kl}^D}{a_{kl} + b_{kl} p_{kl}^D} \right). \quad (9)$$

Furthermore, we can rewrite (2) as

$$\begin{aligned} R_l^D(p^C, p^D, \rho) &= \sum_{k \in \mathcal{K}} \rho_{kl} R_{kl}^D(p^C, p^D, \rho) \\ &= \sum_{k \in \mathcal{K}} \rho_{kl} \log \left( 1 + \frac{\rho_{kl} p_{kl}^D h_l^D}{\sigma_N^2 + p_k^C h_{kl}^{CD}} \right) \end{aligned}$$

$$\begin{aligned} &= \sum_{k \in \mathcal{K}} \log \left( 1 + \frac{\rho_{kl} p_{kl}^D h_l^D}{\sigma_N^2 + p_k^C h_{kl}^{CD}} \right) \\ &= \sum_{k \in \mathcal{K}} \log \left( 1 + \frac{\rho_{kl} p_{kl}^D}{a_{kl} + b_{kl} p_{kl}^D} \right) \\ &\triangleq \hat{R}_l^D(p^D, \rho). \end{aligned}$$

This equation always holds because  $\rho_{kl}$  is a binary variable.

#### REFERENCES

- [1] K. Doppler, M. Rinne, C. Wijting, C. B. Ribeiro, and K. Hugl, "Device-to-device communication as an underlay to LTE-Advanced networks," *IEEE Commun. Mag.*, vol. 47, no. 12, pp. 42–49, Dec. 2009.
- [2] G. Fodor *et al.*, "Design aspects of network assisted device-to-device communications," *IEEE Commun. Mag.*, vol. 50, no. 3, pp. 170–177, Mar. 2012.
- [3] J. Liu, N. Kato, H. Ujikawa, and K. Suzuki, "Device-to-device communication for mobile multimedia in emerging 5G networks," *ACM Trans. Multimed. Comput. Comm. Appl.*, vol. 12, no. 5 s, Article 75, Dec. 2016.
- [4] A. Asadi, Q. Wang, and V. Mancuso, "A survey on device-to-device communication in cellular networks," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 4, pp. 1801–1819, Oct.–Dec., 2014.
- [5] J. Liu, N. Kato, J. Ma, and N. Kadowaki, "Device-to-device communication in LTE-advanced networks: A survey," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 1923–1940, Oct.–Dec., 2015.
- [6] A. H. Land and A. G. Doig, "An automatic method of solving discrete programming problems," *Econometrica, J. Econometric Soc.*, vol. 28, no. 3, pp. 497–520, Jul. 1960.
- [7] R. Yin, G. Yu, H. Zhang, Z. Zhang, and G. Y. Li, "Pricing-based interference coordination for D2D communications in cellular networks," *IEEE Trans. Wireless Commun.*, vol. 14, no. 3, pp. 1519–1532, Mar. 2015.
- [8] H. Nguyen, M. Hasegawa, and W. Hwang, "Distributed resource allocation for D2D communications underlay cellular networks," *IEEE Commun. Lett.*, vol. 20, no. 5, pp. 942–945, May 2016.
- [9] H. Zhang, L. Song, and Z. Han, "Radio resource allocation for device-to-device underlay communication using hypergraph theory," *IEEE Trans. Wireless Commun.*, vol. 15, no. 7, pp. 4852–4861, Jul. 2016.
- [10] R. Zhang, X. Cheng, L. Yang, and B. Jiao, "Interference-aware graph based resource sharing for device-to-device communications underlaying cellular networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, Shanghai, China, Apr. 2013, pp. 140–145.
- [11] D. Feng, L. Lu, Y. Yuan-Wu, G. Y. Li, G. Feng, and S. Li, "Device-to-device communications underlaying cellular networks," *IEEE Trans. Commun.*, vol. 61, no. 8, pp. 3541–3551, Aug. 2013.
- [12] Y. Jiang, Q. Liu, F. Zheng, X. Gao, and X. You, "Energy efficient joint resource allocation and power control for D2D communications," *IEEE Trans. Veh. Technol.*, vol. 65, no. 8, pp. 6119–6127, Aug. 2016.
- [13] H. Sun, X. Chen, Q. Shi, M. Hong, X. Fu, and N. D. Sidiropoulos, "Learning to optimize: Training deep neural networks for interference management," *IEEE Trans. Signal Process.*, vol. 66, no. 20, pp. 5438–5453, Oct. 2018.
- [14] W. Lee, M. Kim, and D. H. Cho, "Deep power control: Transmit power control scheme based on convolutional neural network," *IEEE Commun. Lett.*, vol. 22, no. 6, pp. 1276–1279, Jun. 2018.
- [15] Y. S. Nasir and D. Guo, "Multi-agent deep reinforcement learning for dynamic power allocation in wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2239–2250, Oct. 2019.
- [16] H. Ye, G. Y. Li, and B.-H. F. Juang, "Deep reinforcement learning based resource allocation for V2V communications," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3163–3173, Apr. 2019.
- [17] X. Chen *et al.*, "Multi-tenant cross-slice resource orchestration: A deep reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 10, pp. 2377–2392, Oct. 2019.
- [18] W. Cui, K. Shen, and W. Yu, "Spatial deep learning for wireless scheduling," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1248–1261, Jun. 2019.
- [19] M. Lee, Y. Xiong, G. Yu, and G. Y. Li, "Deep neural networks for linear sum assignment problems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 6, pp. 962–965, Dec. 2018.
- [20] H. He, H. Daume III, and J. M. Eisner, "Learning to search in branch and bound algorithms," in *Proc. Adv. Neural Inform. Process. Syst.*, pp. 3293–3301, Dec. 2014.

- [21] Y. Shen, Y. Shi, J. Zhang, and K. B. Letaief, "LORM: Learning to optimize for resource management in wireless networks with few training samples," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 665–679, Jan. 2020.
- [22] H. Min, J. Lee, S. Park, and D. Hong, "Capacity enhancement using an interference limited area for device-to-device uplink underlying cellular networks," *IEEE Trans. Wireless Commun.*, vol. 10, no. 12, pp. 3995–4000, Dec. 2011.
- [23] G. Yu, L. Xu, D. Feng, R. Yin, G. Y. Li, and Y. Jiang, "Joint mode selection and resource allocation for device-to-device communications," *IEEE Trans. Commun.*, vol. 62, no. 11, pp. 3814–3824, Nov. 2014.
- [24] T. D. Hoang, L. B. Le, and T. Le-Ngoc, "Energy-efficient resource allocation for D2D communications in cellular networks," *IEEE Trans. Veh. Technol.*, vol. 65, no. 9, pp. 6972–6986, Sep. 2016.
- [25] J. Liu, H. Nishiyama, N. Kato, and J. Guo, "On the outage probability of device-to-device communication enabled multi-channel cellular networks: A RSS threshold-based perspective," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 1, pp. 163–175, Jan. 2016.
- [26] J. Liu, S. Zhang, N. Kato, H. Ujikawa, and K. Suzuki, "Device-to-device communications for enhancing quality of experience in software defined multi-tier LTE-A networks," *IEEE Netw.*, vol. 29, no. 4, pp. 46–52, Jul.-Aug. 2015.
- [27] J. Lee and S. Leyffer, *Mixed Integer Nonlinear Programming*. The IMA Volumes in mathematics and its applications, Springer-US, Dec. 2011.
- [28] A. Attia and S. Dayan, "Global overview of imitation learning," *arXiv:1801.06503v1*, Jan. 2018.
- [29] C. Corinna and V. Vapnik, "Support-vector networks," *Mach. Learning*, vol. 20, no. 3, pp. 273–297, Sep. 1995.
- [30] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. Int. Conf. Artificial Intell. Stat.*, pp. 627–635, Apr. 2011.
- [31] C. Chang and C. Lin, "Libsvm: A library for support vector machines," *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, article 27, Apr. 2011.
- [32] J. Song *et al.*, "Learning to search via self-imitation with application to risk-aware planning," in *Proc. Adv. Neural Inform. Process. Syst.*, Dec. 2017.



**Geoffrey Ye Li** (S'93–M'15–SM'97–F'06) received the B.S.E. and M.S.E. degrees from the Nanjing Institute of Technology, Nanjing, China, in 1983 and 1986, respectively, and the Ph.D. degree from Auburn University, Auburn, AL, USA, in 1994. He was a Teaching Assistant and then a Lecturer with Southeast University, Nanjing, China, from 1986 to 1991, a Research and Teaching Assistant with Auburn University, from 1991 to 1994, and a Postdoctoral Research Associate with the University of Maryland at College Park, College Park, MD, USA, from 1994 to 1996. He was with AT&T Labs-Research, Red Bank, NJ, USA, as a Senior and then a Principal Technical Staff Member from 1996 to 2000. Since 2000, he has been with the School of Electrical and Computer Engineering, Georgia Institute of Technology, as an Associate Professor and then a Full Professor. His general research interests include statistical signal processing and machine learning for wireless communications. In these areas, he has published over 500 journal and conference papers in addition to over 40 granted patents. His publications have been cited over 38,000 times and he has been recognized as the *World's Most Influential Scientific Mind*, also known as a *Highly-Cited Researcher*, by Thomson Reuters almost every year. He was awarded IEEE Fellow for his contributions to *signal processing for wireless communications* in 2005. He won 2010 IEEE ComSoc Stephen O. Rice Prize Paper Award, 2013 IEEE VTS James Evans Avant Garde Award, 2014 IEEE VTS Jack Neubauer Memorial Award, 2017 IEEE ComSoc Award for Advances in Communication, and 2017 IEEE SPS Donald G. Fink Overview Paper Award. He also received 2015 Distinguished Faculty Achievement Award from the School of Electrical and Computer Engineering, Georgia Tech. He has been involved in editorial activities for over 20 technical journals for the IEEE, including founding Editor-in-Chief of IEEE 5G Tech Focus. He has organized and chaired many international conferences, including Technical Program Vice-Chair of IEEE ICC'03, Technical Program Co-Chair of IEEE SPAWC'11, General Chair of IEEE GlobalSIP'14, Technical Program Co-Chair of IEEE VTC'16 (Spring), and General Co-Chair of IEEE VTC'19 (Fall).



**Mengyuan Lee** received the B.S.E degree in information engineering from Zhejiang University, Hangzhou, China, in 2018. She is currently working toward the Ph.D degree with the College of Information Science and Electronic Engineering, Zhejiang University, Hangzhou, China. She is also a Visiting Student at the North Carolina State University, NC, USA. Her current research interests mainly include machine learning and wireless communications. She received Chu Kochen Scholarship, the highest honor at Zhejiang University, in 2017.



**Guanding Yu** (S'05–M'07–SM'13) received the B.E. and Ph.D. degrees in communication engineering from Zhejiang University, Hangzhou, China, in 2001 and 2006, respectively. He joined Zhejiang University in 2006, and is now a Full Professor with the College of Information and Electronic Engineering. From 2013 to 2015, he was also a Visiting Professor at the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, USA. His research interests include 5G communications and networks, mobile edge computing, and machine

learning for wireless networks. Dr. Yu has served as a Guest Editor of IEEE COMMUNICATIONS MAGAZINE Special Issue on Full-Duplex Communications, an Editor of IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS Series on Green Communications and Networking, and a Lead Guest Editor of IEEE WIRELESS COMMUNICATIONS MAGAZINE Special Issue on LTE in Unlicensed Spectrum, and an Editor of IEEE ACCESS. He is now serving as an Editor of IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING and an Editor of IEEE WIRELESS COMMUNICATIONS LETTERS. He received the 2016 IEEE ComSoc Asia-Pacific Outstanding Young Researcher Award. He regularly sits on the technical program committee (TPC) boards of prominent IEEE conferences such as ICC, GLOBECOM, and VTC. He also serves as a Symposium Co-Chair for IEEE Globecom 2019 and a Track Chair for IEEE VTC 2019'Fall.